

Predicting NBA Playoff Underperformance

A game-level ML system with live API and web app

PRESENTED BY

Hanzhang Yuan

DATE

June 2, 2026

LIVE DEPLOYED SERVICES

Web App (Google Cloud Run)
nba-playoff-app-803317660037
.us-central1.run.app

REST API (Google Cloud Run)
nba-playoff-api-803317660037
.us-central1.run.app

Why Some Stars Fade in the Playoffs

THE PROBLEM

Given only data available before tip-off, can we predict whether an NBA player will underperform their regular-season Game Score baseline in a specific playoff game?

WHY IT MATTERS

Playoff basketball is a different game.

Defensive intensity rises, rotations shrink from 10 to 8 players, and opposing coaches scout individual tendencies. Quantifying fade-risk before tip-off has real value for fans, analysts, and front offices.

KEY OBJECTIVES

1. Build a real ML pipeline (scrape → features → model → API → App) 2. Deploy a usable system 3. Be honest about results

PROBLEM TYPE

Binary

Underperform yes / no

UNIT OF ANALYSIS

Player-game

Not player-season

DATA RANGE

22 seasons

2003-04 to 2024-25

Scraping 22 Seasons of NBA Box Scores

DATA SOURCE

NBA Stats API via `nba_api`

Player game logs (regular + playoff) and team advanced stats for 2003-04 through 2024-25.

PREPROCESSING PIPELINE

- | | |
|--------------|--|
| 1. Scrape | 1.2s rate limit + 5-retry backoff + disk cache (resumable) |
| 2. Parse | JSON → Parquet with Hollinger Game Score computed |
| 3. Aggregate | Per-season per-player + per-team-defense stats |
| 4. Engineer | 45 features, z-scored, expanding window for history |

REQUESTS

~25,000
across 22 seasons

PLAYER-GAMES

575,801
regular + playoff

MODELING SET

30,189
filtered playoff rows

CHALLENGES SOLVED

Era drift → per-season z-score | **Connection resets** → `nba_api` 1.11 + retry/backoff | **Heteroskedastic baselines** → relative 20% threshold

Three Models, 45 Features, Strict Time Split

MODELS COMPARED (held-out 2023-25 test set)

BASILINE

Logistic Regression

Test AUC: 0.650

F1: 0.638

MID-TIER

Random Forest

Test AUC: 0.650

F1: 0.621

PRIMARY

XGBoost

Test AUC: 0.649

F1: 0.632

Optuna 50 trials · TimeSeriesSplit CV

TOP SHAP FEATURES



45 FEATURES · 6 FAMILIES

- A. Player regular-season baseline (15)
- B. Player playoff history (10)
- C. Game context (8)
- D. Opponent strength (5)
- E. Player × opponent matchup (4)
- F. In-series momentum (3)

Train 2003-2020 Val 2021-23 Test 2023-25 · AUC 0.65 meets pre-registered acceptance threshold ✓

Deployed System Architecture



TECHNOLOGY STACK

Data	nba_api · pandas · pyarrow (Parquet)
Modeling	scikit-learn · XGBoost · Optuna · SHAP
Backend	Flask + flask-restx · gunicorn · marshmallow
Frontend	Streamlit
Container	Docker · docker-compose
Deployment	Google Cloud Run (both API + App) · Streamlit Cloud backup
CI / CD	GitHub Actions · pytest

Two Modes • Replay & What-if

REPLAY MODE

Real game • Model vs reality

Pick any actual playoff game from 2023-24 or 2024-25. The model produces its pre-game prediction; the app then shows what really happened.

FEATURES:

- ▶ Cascading dropdowns: season → player → opponent → game
- ▶ Player list restricted to actual playoff participants
- ▶ Opponent list restricted to that player's real matchups
- ▶ Side-by-side model prediction vs actual Game Score
- ▶ Green/red badge: "Model called it correctly?"

WHAT-IF MODE

Predict the future

Pick any player, any opponent, any series situation. Use 2025-26 regular-season stats to predict playoffs that haven't happened yet, or ask hypothetical questions.

FEATURES:

- ▶ Any player from 5,126 in the lookup table
- ▶ Any of 30 NBA teams as opponent
- ▶ Hypothetical matchups (Jokic vs 1996 Bulls)
- ▶ 2025-26 ongoing playoff prediction with current stats
- ▶ Advanced sliders to override player stats for what-ifs

SHARED FEATURES (BOTH MODES)

Probability + risk badge

% chance + color-coded risk level

Predicted Game Score

Expected vs RS average, with delta

Top 3 SHAP factors

Plain-English explanations of risk drivers

Challenges, AI Assistants, Lessons, Next Steps

TOP TECHNICAL CHALLENGES

Data leakage in expanding-window features

Wrote 7 pytest assertions BEFORE the pipeline → caught 2 real bugs

nba_api connection resets from corporate network

Upgraded to nba_api 1.11+ with patched headers + retry/backoff

Heteroskedastic Game Score baselines

Relative 20% threshold + 1.0 GS floor → +3.8 pp test AUC

API returning identical predictions for all inputs

Recompute series-momentum from real games 1..N-1 at lookup time

AI ASSISTANTS IN DEVELOPMENT

Claude (primary)

Architecture brainstorming · code scaffolding · debugging the no-leakage pipeline · slide design

HIGHEST-VALUE INTERACTION

Surfacing that the proposal's target definition (absolute 1.0 GS threshold) was statistically wrong → switching to relative 20% threshold added 3.8 pp test AUC.

WHERE AI OUTPUT NEEDED HEAVY MODIFICATION

First version of feature_lookup returned NaN for nearly every feature, producing identical predictions across all inputs. Required complete rewrite to base lookups on the real training table.

KEY LESSON: Problem definition (+3.8 pp) mattered more than model choice (~0.1 pp).

FUTURE WORK: scheduled weekly scrape · injury features · series-level model · 2025-26 in-progress predictions